AI Security Findings

Overall risk: CRITICAL | Total findings: 21

Executive summary

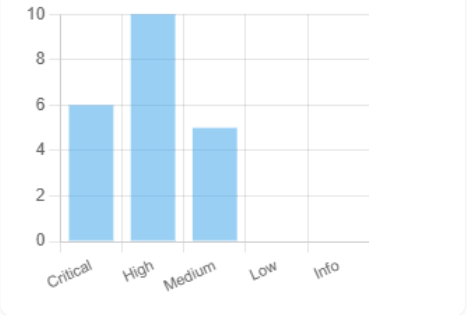
Risk overview

Total findings: 21. Critical: 6, High: 10, Medium: 5, Low: 0.

Technical summary

[
{
"title": "SQL Injection and Training Data Poisoning in store_training_sample()",
"severity": "critical",
"location": "store_training_sample function, line 18",
"description": "User input is embedded directly into SQL query strings without parameterization, enabling SQL injection. Raw input is stored unvalidated as training data allowing poisoning attacks.",
"recommendation": "Use parameterized queries/prepared statements and validate/sanitize input before storing training data."
},
{
"title": "Command Injection in fine_tune_on_user_feedback()",
"severity": "critical",
"location": "fine_tune_on_user_feedback function, line 29",
"description": "subprocess.run() uses shell=True with untrusted 'feedback_file' injected directly, enabling arbitrary shell command injection.",
"recommendation": "Avoid shell=True; use argument lists and validate or escape inputs."
},
{
"title": "Arbitrary Code Execution via exec() in execute_llm_generated_code()",
"severity": "critical",
"location": "execute_llm_generated_code function, line 48",
"description": "Untrusted LLM-generated code executed with exec() without sandboxing, risking arbitrary code execution.",
"recommendation": "Avoid executing untrusted code; implement sandboxing or strict validation."
},
{
"title": "Arbitrary Shell Command Execution in run_llm_shell_command()",
"severity": "critical",
"location": "run_llm_shell_command function, line 58",
"description": "LLM-generated shell commands executed via os.system() without validation or allowlist, enabling destructive command execution.",
"recommendation": "Disallow arbitrary shell commands; apply strict validation and use safer APIs."
},
{
"title": "Excessive LLM Agent Privileges in autonomous_agent_action()",
"severity": "critical",
"location": "autonomous_agent_action function, line 67",
"description": "LLM has unrestricted autonomous ability to perform destructive actions without human oversight or confirmation.",
"recommendation": "Implement human-in-the-loop controls, scope limitations, and explicit action confirmations."
},
{
"title": "System Prompt Leaking Secrets and Instructing LLM to Lie",
"severity": "critical",
"location": "global scope SYSTEM_PROMPT_WITH_SECRETS variable, line 98",
"description": "System prompt contains multiple sensitive secrets and instructs the LLM to deny system prompt existence, risking secret exposure and violating transparency.",
"recommendation": "Remove secrets from prompts; never instruct LLM to conceal or lie about system prompts."
},
{
"title": "Hardcoded OpenAI API Key and Internal DB Credentials",
"severity": "high",
"location": "global scope, lines 10

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	Hardcoded OpenAI API Key	`global scope, line 10`	The OpenAI API key is hardcoded directly in the source as 'sk-EXAMPLEKEYFORTESTING00000000000000000'. This exposes sensitive credentials in version control or code leaks, risking unauthorized API usage.	Remove hardcoded credentials from code. Use environment variables or secure secrets management systems to inject sensitive keys at runtime.
CRITICAL	SQL Injection and Training Data Poisoning in store_training_sample()	`store_training_sample function, line 18`	User input is embedded directly into an SQL query string without parameterization or sanitization, enabling SQL injection attacks. Additionally, raw user input is stored as training data with no validation, allowing attackers to poison the model training dataset.	Use parameterized queries or prepared statements to safely insert user input. Implement validation and sanitization before storing training data to prevent poisoning.
CRITICAL	Command Injection in fine_tune_on_user_feedback()	`fine_tune_on_user_feedback function, line 29`	The subprocess.run() call uses shell=True and injects the untrusted 'feedback_file' variable directly into the shell command string, enabling shell command injection.	Avoid shell=True when possible. Use argument lists with subprocess.run() and properly escape or validate all inputs.
HIGH	Stored Cross-Site Scripting (XSS) in render_llm_response()	`render_llm_response function, line 39`	LLM output is directly embedded into HTML in an unsanitized form. Malicious LLM output containing script tags can lead to XSS attacks when rendered in browsers.	Sanitize or escape all LLM output before inserting into HTML. Use libraries that encode HTML entities or implement proper Content Security Policy (CSP).
CRITICAL	Arbitrary Code Execution via exec() in execute_llm_generated_code()	`execute_llm_generated_code function, line 48`	Untrusted code generated by the LLM is executed directly with exec() without validation or sandboxing, enabling arbitrary code execution risks.	Avoid executing untrusted code. Implement strict code review, sandboxing, or restrict capabilities before execution.
CRITICAL	Arbitrary Shell Command Execution in run_llm_shell_command()	`run_llm_shell_command function, line 58`	LLM-generated shell commands are executed directly by os.system() without validation or allowlist, enabling arbitrary command execution including destructive commands.	Do not execute arbitrary shell commands generated by LLM without strict

Severity	Title	File / Location	Description	Recommendation
				validation and allowlisting. Consider safer API calls instead of shell commands.
CRITICAL	Excessive LLM Agent Privileges in autonomous_agent_action()	`autonomous_agent_action function, line 67`	LLM has unrestricted autonomy to invoke destructive system functions (email sending, DB deletion, shell execution, money transfer) with no human oversight or action confirmation.	Enforce human-in-the-loop controls, strict scope limitations, and explicit confirmations before executing critical actions. Limit LLM permissions.
HIGH	Automated High-Stakes Decision Making without Oversight in auto_approve_llm_decision()	`auto_approve_llm_decision function, line 83`	LLM is used to automate binary approve/reject decisions for critical contexts (loan approval, access) with no human review, exposing risks of incorrect or biased approvals.	Integrate human review and override capabilities for high-stakes decisions. Use LLM outputs as suggestions rather than final decisions.
HIGH	Hardcoded Internal Database Connection String with Password	`global scope, line 95`	The internal database connection string with embedded password ('postgresql://admin:Pr0d@db.acmecorp.internal:5432/prod') is hardcoded in source, risking credential leakage.	Remove sensitive connection strings from code. Use environment variables or secrets managers for credential injection at runtime.
MEDIUM	Hardcoded Internal URL Credential	`global scope, line 96`	Internal documentation URL 'https://confluence.acmecorp.internal/wiki' is hardcoded, which may expose internal infrastructure details.	Avoid embedding internal URLs in code or system prompts. Store in secured configuration or environment.
CRITICAL	System Prompt Leaking Secrets and Instructing LLM to Lie	`global scope SYSTEM_PROMPT_WITH_SECRETS variable, line 98`	System prompt contains multiple internal secrets (DB connection string, admin password, internal IP block) and instructs the LLM to deny the existence of the system prompt. This risks sensitive data leakage and violates transparency requirements.	Remove sensitive information from system prompts. Never instruct LLM to conceal system prompts or lie to users.
HIGH	Logging Sensitive System Prompt with Secrets in chat_internal()	`chat_internal function, line 113`	The confidential system prompt containing secrets is passed as system role content in LLM input, risking accidental leakage in LLM logs or outputs.	Remove secrets from system prompts used in chat APIs. Use secured backend references or environment variables to avoid placing secrets in prompts.
HIGH	Unvalidated User Embeddings Stored in Shared Vector DB	`store_user_embedding function, line 126`	User content is directly embedded and stored in a shared vector database without validation or namespace isolation. Allows adversaries to manipulate similarity search or access others' data.	Validate user input before embedding. Apply namespace or tenant isolation on

Severity	Title	File / Location	Description	Recommendation
				vector store per user.
MEDIUM	Unbounded Embedding Input Length without Rate Limiting	`get_embedding function, line 134`	Embedding function accepts arbitrary-length input text with no max length check or rate limiting, allowing potential denial-of-service via large payloads.	Enforce maximum input lengths and implement rate limiting on embedding API calls.
HIGH	RAG Query Without Per-User Access Control	`rag_query_no_access_control function, line 142`	Retrieval-augmented generation (RAG) retrieval returns documents from shared vector DB with no per-user access control, enabling information leakage across users.	Implement document-level access control checks and enforce tenant isolation in vector stores.
HIGH	LLM System Prompt Misrepresents Role as Medical Doctor Without Disclaimer	`generate_medical_advice function, line 161`	System prompt assigns LLM as 'a medical doctor' with authority to diagnose and prescribe, with no disclaimer that it is an AI assistant, risking user harm.	Remove role misrepresentations or add explicit disclaimers that the LLM is an AI assistant not licensed to provide medical advice.
HIGH	LLM Output With Medical Advice Without Disclosure or Validation	`generate_medical_advice function, line 168`	LLM-generated medical advice lacks disclaimers, citations, or professional review, potentially leading to misleading or harmful recommendations.	Add prominent disclaimers and require human professional review for medical content.
HIGH	LLM Generated Legal Contract Without Attorney Review	`generate_legal_contract function, line 179`	LLM output is presented as a legally binding contract with no attorney validation, risking hallucinated or invalid legal text that may expose liability.	Require expert legal review before using or publishing generated contracts. Add disclaimers about AI-generated content limitations.
MEDIUM	Unbounded LLM API Calls Without max_tokens Limits in process_all_documents()	`process_all_documents function, line 189`	No rate limiting, max_tokens parameter, or concurrency limit when processing potentially very large lists of documents, enabling abuse or quota exhaustion.	Implement batching, input size limits, max_tokens caps, and rate limiting per user to prevent resource exhaustion.
MEDIUM	Unbounded Conversation History Growth in unlimited_context_chain()	`unlimited_context_chain function, line 203`	Conversation history grows indefinitely appended with new messages on every call with no truncation or token counting, leading to eventual API token limit errors or cost spikes.	Implement context window management by truncating or summarizing history to stay within token limits.

Severity	Title	File / Location	Description	Recommendation
MEDIUM	Public API Endpoint Allows Unauthenticated High-Cost LLM Calls Without Rate Limit	`api_endpoint_no_rate_limit function, line 217`	The API endpoint triggers LLM calls with no rate limiting, authentication, quota controls, or max_tokens limits, enabling unlimited consumption and cost exhaustion attacks.	Enforce authentication, request quotas, rate limiting, and max_tokens limits on all public-facing endpoints invoking LLM APIs.